Introduction on Object in PHP

Objects are real-world entities. Objects are defined from classes in Object-Oriented Programming like PHP. When a class is defined, we can create many objects out of the class. Example Class Car is defined, then Mercedes, BMW, Skoda are all objects of the Class Car.

A class is a blueprint of an object. A class contains variables and functions. These data variables are called properties and data functions are called data methods.

The definition of an object goes like this, An object is an instance of a class. We can create an instance of the class by using the new keyword. We can create multiple instances of the class. These instances can now access the class functions, the class members.

How to Create an Object?

Creating an object is the same as instantiating a class. This instance is created using the new keyword. This process is called instantiation**.** Since objects are instances of a class and can be created using a new keyword let us take a look at how these instances are created.

Syntax:

```
objectname = new Classname();
```

Examples:

```
$parrot = new Bird();

$pigeon = new Bird();
$woodpecker = new Bird();
```

Above are three different objects of the class Bird. Using these objects we can access the properties and functions of the class Bird.

<u>What is a new keyword?</u>

When we instantiate a class, we are actually creating an object of the class. To create object it is essential to use a new keyword. While using a constructor in a class, the constructor is called automatically when the object is initialized using the new keyword.

Properties of Object

Properties are variables that are defined within a class. These variables are then used by the methods, objects of the class. These variables can be public, protected or private. By default, the public is used. The value of a variable may or may not contain a default value, meaning that the variable may be initialized with a value or not.

The variable names are case sensitive, meaning that $name is different from $Name. There is a naming convention like if the variable contains more than one word than the second word will start with a capital letter like $firstName, $lastName and so on.

Let us look at the below program to understand the properties.

```
class Birds {
public $birdsFly = 'sky';
public $birdsSound = 'vocal';
public $birdsBuildNests ='trees';
}
```

The following program explains how to declare a class Bird and its properties like a bird flies make sound and build a nest and a method on what a bird does.

Code:

```
<?php
//example to access properties of a class
class Birds {
// properties
```

```
public $birdsFly = 'sky';

public $birdsSound = 'vocal';

public $birdsBuildNests = 'trees';

//methods

public function birdDoes()

{

echo 'Bird';

}

}

//object of class is declared

$obj = new Birds();

//properties of class Bird are accessed using object

echo '<br>  Bird Flies =  '.$obj->birdsFly;

echo '<br>  Bird Makes Sound = '.$obj->birdsSound;

echo '<br>  Bird Build Nests = '.$obj->birdsBuildNests;
?>
```

Output:

Bird Flies = sky

Bird Makes Sound = vocal

Birds Build Nests = trees

Methods of Object in PHP

As the properties of a class, we can define member functions in a class. These functions can then be called from an object. These functions are called as methods of a class. These functions can

be public, private or protected. By default is public. Also while declaring the function we declare it as

Syntax:

```
public function functionaname() {

//statements
}
```

Example:

```
class Bird {

public function makesSound() {

// statements

}

public functions looksForFood() {

// statements

}

}

$obj = new Bird;

echo $bird->makesSound();
echo $bird->looksForFood();
```

Code:

```
<?php

//example to access methods of a class

class Birds {

// properties

public $birdsFly;
```

```
public $birdsBuildNests;

//method 1  - set Method1

public function set_birdFlies($input) {

$this->birdsFly = $input ;

}

//method 1 - get Method1

public function get_birdFlies() {

return $this->birdsFly;

}

//method 2  - set Method2

public function set_BirdBuildsNest($input) {

$this->birdsBuildNests = $input ;

}

//method 2 - get Method2

public function get_BirdBuildsNest() {

return $this->birdsBuildNests;

}

}

//object of class is declared

$obj = new Birds();

$obj->set_birdFlies('Fly');

echo '<br>  Bird Flies =  '.$obj->get_birdFlies();

$obj->set_BirdBuildsNest('Trees');
echo '<br>  Bird Builds Nest =  '.$obj->get_BirdBuildsNest();?>
```

Output:

Bird Flies = Fly

Bird Builds Nest = Trees

Object and Constructors

A constructor is a special method. When a new object is created this method is invoked automatically. There is no need for calling the method explicitly from an object.

Syntax:

```
__construct();// double underscores are used.
```

Suppose there are two classes one base class and the other is derived class. If the derived class do not have its own constructor and wants to inherit the base class constructor we need to declare it in the following syntax:

Syntax:

```
parent::__construct();
```

Code:

```
<?php
//example to use constructor in a class
class Birds {
// properties
public $makesSound;
// the constructor is called when object is created
public function __construct($input) {
$this->makesSound = $input;
echo 'Bird makes Sound: '.$this->makesSound;
}
}
```

```
//object of class is declared

$obj = new Birds('Vocal');
?>
```

Output:

Bird makes Sound: Vocal

Constructor and Object with Inheritance

In the below program we will see that how objects call the base class constructor is internally by creating an object. Since inheritance is used we can use parent:: __construct() to call the base class constructor.

Code:

```
<?php

//example to use constructor and object of a class

class Birds {

// properties

public $bird_flies='sky';

// the constructor is called when object is created

public function __construct() {

echo 'In the Base Class Constructor';

}

}

class EagleBird extends Birds {

function __construct() {

parent::__construct();

}

public function BirdDoes() {
```

```
return $this->bird_flies;

}

}

//object of class is declared

$obj = new EagleBird();

echo '<br> Method : ' .$obj->BirdDoes();
?>
```

Output:

In the Base Class Constructor

Method: sky

Accessing Properties and Methods

Once you have an object, you can use the -> notation to access methods and proper ties of the object:

$object->propertyname $object->methodname([arg, ...])

For example:

echo "Moana is {$moana->age} years old.\n"; // property access

$moana->birthday(); // method call

$moana->setAge(21); // method call with arguments

Methods act the same as functions (only specifically to the object in question), so they can take arguments and return a value:

$clan = $moana->family("extended");

Within a class's definition, you can specify which methods and properties are publicly accessible and which are accessible only from within the class itself using the public and private access modifiers. You can use these to provide encapsulation.

You can use variable variables with property names:

```
$prop = 'age';
echo $moana->$prop;
```

A static method is one that is called on a class, not on an object. Such methods cannot access properties. The name of a static method is the class name followed by two colons and the function name. For instance, this calls the p() static method in the HTML class:

```
HTML::p("Hello, world");
```

When declaring a class, you define which properties and methods are static using the static access property.Once created, objects are passed by reference—that is, instead of copying around the entire object itself (a time- and memory-consuming endeavor), a reference to the object is passed around instead.

For example:

```
$f = new Person("Pua", 75);
$b = $f; // $b and $f point at same object
$b->setName("Hei Hei");
printf("%s and %s are best friends.\n", $b->getName(), $f->getName());
```

Hei Hei and Hei Hei are best friends.

If you want to create a true copy of an object, you use the clone operator:

```
$f = new Person("Pua", 35);
$b = clone $f; // make a copy
$b->setName("Hei Hei");// change the copy
printf("%s and %s are best friends.\n", $b->getName(), $f->getName());
```

Pua and Hei Hei are best friends.

When you use the clone operator to create a copy of an object and that class declares the __clone() method, that method is called on the new object immediately after it's cloned. You might use this in cases where an object holds

external resources (such as file handles) to create new resources, rather than copying the existing ones.

Declaring a Class

To design your program or code library in an object-oriented fashion, you'll need to define your own classes***, using the class keyword***. *A class definition includes the class name and the properties and methods of the class.* **Class names are case insensitive and must conform to the rules for PHP identifiers.** Among others, the class name stdClass is reserved. Here's the syntax for a class definition:

```
class classname [ extends baseclass ] [ implements interfacename ,
[interfacename, ... ] ] {
[ use traitname, [ traitname, ... ]; ]
[ visibility $property [ = value ]; ... ]
[ function functionname (args) [: type ] {
// code
}
...
]
}
```

Declaring Methods

A method is a function defined inside a class. Although PHP imposes no special restrictions, most methods act only on data within the object in which the method resides. Method names beginning with two underscores (__) may be used in the future by PHP (and are currently used for the object serialization methods __sleep() and __wakeup(), described later in this chapter, among others), so it's recommended that you do not begin your method names with this sequence.

Within a method, the $this variable contains a reference to the object on which the method was called. For instance, if you call $moana->birthday(), inside the birth day() method, $this holds the same value as $moana. Methods use the $this variable

to access the properties of the current object and to call other methods on that object. Here's a simple class definition of the Person class that shows the $this variable in action:

```
class Person {
public $name = '';
function getName() {
return $this->name;
}
function setName($newName) {
$this->name = $newName;
}
}
```

As you can see, the getName() and setName() methods use $this to access and set the $name property of the current object.

To declare a method as a static method, use the static keyword. Inside of static methods the variable $this is not defined. For example:

```
class HTMLStuff {
static function startTable() {
echo "<table border=\"1\">\n";
}
static function endTable() {
echo "</table>\n";
}
}
HTMLStuff::startTable();
// print HTML table rows and columns
HTMLStuff::endTable();
```

If you declare a method using the final keyword, subclasses cannot override that method. For example:

```
class Person {
public $name;
```

```
final function getName() {
return $this->name;
}
}
class Child extends Person {
// syntax error
function getName() {
// do something
}
}
```

Using access modifiers, you can change the visibility of methods. Methods that are accessible outside methods on the object should be declared public; methods on an instance that can be called only by methods within the same class should be declared private. Finally, methods declared as protected can be called only from within the object's class methods and the class methods of classes inheriting from the class. Defining the visibility of class methods is optional; if a visibility is not specified, a method is public. For example, you might define:

```
class Person {
public $age;
public function __construct() {
$this->age = 0;
```



```
}
public function incrementAge() {
$this->age += 1;
$this->ageChanged();
}
protected function decrementAge() {
$this->age -= 1;
$this->ageChanged();
}
```

```
private function ageChanged() {
echo "Age changed to {$this->age}";
}
}
class SupernaturalPerson extends Person {
public function incrementAge() {
// ages in reverse
$this->decrementAge();
}
}
$person = new Person;
$person->incrementAge();
$person->decrementAge(); // not allowed
$person->ageChanged(); // also not allowed
$person = new SupernaturalPerson;
$person->incrementAge(); // calls decrementAge under the hood
```

You can use type hinting (described in Chapter 3) when declaring a method on an object:

```
class Person {
function takeJob(Job $job) {
echo "Now employed as a {$job->title}\n";
}
}
```

When a method returns a value, you can use type hinting to declare the method's return value type:

```
class Person {
function bestJob(): Job {
$job = Job("PHP developer");
return $job;
}
}
```

PHP - The $this Keyword

The $this keyword refers to the current object, and is only available inside methods.

Look at the following example:

Example

```
<?php
class Fruit {
  public $name;
}
$apple = new Fruit();
?>
```

So, where can we change the value of the $name property? There are two ways:

1. Inside the class (by adding a set_name() method and use $this):

Example

```
<?php
class Fruit {
  public $name;
  function set_name($name) {
    $this->name = $name;
  }
}
$apple = new Fruit();
$apple->set_name("Apple");

echo $apple->name;
?>
```

2. Outside the class (by directly changing the property value):

Example

```
<?php
class Fruit {
```

```
  public $name;
}
$apple = new Fruit();
$apple->name = "Apple";

echo $apple->name;
?>
```

PHP - instanceof

You can use the `instanceof` keyword to check if an object belongs to a specific class:

Example

```
<?php
$apple = new Fruit();
var_dump($apple instanceof Fruit);
?>
```

PHP - The __construct Function

A constructor allows you to initialize an object's properties upon creation of the object.

If you create a `__construct()` function, PHP will automatically call this function when you create an object from a class.

Notice that the construct function starts with two underscores (__)!

We see in the example below, that using a constructor saves us from calling the set_name() method which reduces the amount of code:

Example

```
<?php
class Fruit {
  public $name;
```

```
  public $color;

  function __construct($name) {
    $this->name = $name;
  }
  function get_name() {
    return $this->name;
  }
}

$apple = new Fruit("Apple");
echo $apple->get_name();
?>
```

Example

```
<?php
class Fruit {
  public $name;
  public $color;

  function __construct($name, $color) {
    $this->name = $name;
    $this->color = $color;
  }
  function get_name() {
    return $this->name;
  }
  function get_color() {
    return $this->color;
  }
}

$apple = new Fruit("Apple", "red");
echo $apple->get_name();
echo "<br>";
echo $apple->get_color();
?>
```

PHP - The __destruct Function

A destructor is called when the object is destructed or the script is stopped or exited.

If you create a `__destruct()` function, PHP will automatically call this function at the end of the script.

Notice that the destruct function starts with two underscores (__)!

The example below has a __construct() function that is automatically called when you create an object from a class, and a __destruct() function that is automatically called at the end of the script:

Example

```
<?php
class Fruit {
  public $name;
  public $color;

  function __construct($name) {
    $this->name = $name;
  }
  function __destruct() {
    echo "The fruit is {$this->name}.";
  }
}

$apple = new Fruit("Apple");
?>
```

Example

```
<?php
class Fruit {
  public $name;
  public $color;

  function __construct($name, $color) {
    $this->name = $name;
    $this->color = $color;
  }
  function __destruct() {
    echo "The fruit is {$this->name} and the color is {$this->color}.";
  }
}

$apple = new Fruit("Apple", "red");
?>
```

PHP - Access Modifiers

Properties and methods can have access modifiers which control where they can be accessed.

There are three access modifiers:

- `public` - the property or method can be accessed from everywhere. This is default
- `protected` - the property or method can be accessed within the class and by classes derived from that class
- `private` - the property or method can ONLY be accessed within the class

In the following example we have added three different access modifiers to three properties (name, color, and weight). Here, if you try to set the name property it will work fine (because the name property is public, and can be accessed from everywhere). However, if you try to set the color or weight property it will result in a fatal error (because the color and weight property are protected and private):

Example

```
<?php
class Fruit {
  public $name;
  protected $color;
  private $weight;
}

$mango = new Fruit();
$mango->name = 'Mango'; // OK
$mango->color = 'Yellow'; // ERROR
$mango->weight = '300'; // ERROR
?>
```

In the next example we have added access modifiers to two functions. Here, if you try to call the set_color() or the set_weight() function it will result in a fatal error (because the two functions are considered protected and private), even if all the properties are public:

Example

```
<?php
class Fruit {
  public $name;
```

```
  public $color;
  public $weight;

  function set_name($n) {  // a public function (default)
    $this->name = $n;
  }
  protected function set_color($n) { // a protected function
    $this->color = $n;
  }
  private function set_weight($n) { // a private function
    $this->weight = $n;
  }
}

$mango = new Fruit();
$mango->set_name('Mango'); // OK
$mango->set_color('Yellow'); // ERROR
$mango->set_weight('300'); // ERROR
?>
```

PHP - What is Inheritance?

Inheritance in OOP = When a class derives from another class.

The child class will inherit all the public and protected properties and methods from the parent class. In addition, it can have its own properties and methods.

An inherited class is defined by using the `extends` keyword.

Let's look at an example:

Example

```
<?php
class Fruit {
  public $name;
  public $color;
  public function __construct($name, $color) {
    $this->name = $name;
    $this->color = $color;
  }
  public function intro() {
    echo "The fruit is {$this->name} and the color is {$this->color}.";
  }
}
```

```
// Strawberry is inherited from Fruit
class Strawberry extends Fruit {
  public function message() {
    echo "Am I a fruit or a berry? ";
  }
}
$strawberry = new Strawberry("Strawberry", "red");
$strawberry->message();
$strawberry->intro();
?>
```

Try it Yourself »

Example Explained

The Strawberry class is inherited from the Fruit class.

This means that the Strawberry class can use the public $name and $color properties as well as the public __construct() and intro() methods from the Fruit class because of inheritance.

The Strawberry class also has its own method: message().

PHP - Inheritance and the Protected Access Modifier

In the previous chapter we learned that protected properties or methods can be accessed within the class and by classes derived from that class. What does that mean?

Let's look at an example:

Example

```
<?php
class Fruit {
  public $name;
  public $color;
  public function __construct($name, $color) {
    $this->name = $name;
    $this->color = $color;
```

```
  }
  protected function intro() {
    echo "The fruit is {$this->name} and the color is {$this->color}.";
  }
}

class Strawberry extends Fruit {
  public function message() {
    echo "Am I a fruit or a berry? ";
  }
}

// Try to call all three methods from outside class
$strawberry = new Strawberry("Strawberry", "red");  // OK. __construct()
is public
$strawberry->message(); // OK. message() is public
$strawberry->intro(); // ERROR. intro() is protected
?>
```

In the example above we see that if we try to call a protected method (intro()) from outside the class, we will receive an error. public methods will work fine!

Let's look at another example:

Example

```
<?php
class Fruit {
  public $name;
  public $color;
  public function __construct($name, $color) {
    $this->name = $name;
    $this->color = $color;
  }
  protected function intro() {
    echo "The fruit is {$this->name} and the color is {$this->color}.";
  }
}

class Strawberry extends Fruit {
  public function message() {
    echo "Am I a fruit or a berry? ";
    // Call protected method from within derived class - OK
    $this -> intro();
  }
}
```

```
$strawberry = new Strawberry("Strawberry", "red"); // OK. __construct() is
public
$strawberry->message(); // OK. message() is public and it calls intro()
(which is protected) from within the derived class
?>
```

In the example above we see that all works fine! It is because we call the protected method (intro()) from inside the derived class.

PHP - Overriding Inherited Methods

Inherited methods can be overridden by redefining the methods (use the same name) in the child class.

Look at the example below. The __construct() and intro() methods in the child class (Strawberry) will override the __construct() and intro() methods in the parent class (Fruit):

Example

```
<?php
class Fruit {
  public $name;
  public $color;
  public function __construct($name, $color) {
    $this->name = $name;
    $this->color = $color;
  }
  public function intro() {
    echo "The fruit is {$this->name} and the color is {$this->color}.";
  }
}

class Strawberry extends Fruit {
  public $weight;
  public function __construct($name, $color, $weight) {
    $this->name = $name;
    $this->color = $color;
    $this->weight = $weight;
  }
  public function intro() {
    echo "The fruit is {$this->name}, the color is {$this->color}, and the
weight is {$this->weight} gram.";
```

```
  }
}

$strawberry = new Strawberry("Strawberry", "red", 50);
$strawberry->intro();
?>
```

PHP - The final Keyword

The `final` keyword can be used to prevent class inheritance or to prevent method overriding.

The following example shows how to prevent class inheritance:

Example

```
<?php
final class Fruit {
  // some code
}

// will result in error
class Strawberry extends Fruit {
  // some code
}
?>
```

The following example shows how to prevent method overriding:

Example

```
<?php
class Fruit {
  final public function intro() {
    // some code
  }
}

class Strawberry extends Fruit {
  // will result in error
  public function intro() {
    // some code
  }
}
```

```
?>
```

PHP - Class Constants

Constants cannot be changed once it is declared.

Class constants can be useful if you need to define some constant data within a class.

A class constant is declared inside a class with the `const` keyword.

Class constants are case-sensitive. However, it is recommended to name the constants in all uppercase letters.

We can access a constant from outside the class by using the class name followed by the scope resolution operator (`::`) followed by the constant name, like here:

Example

```
<?php
class Goodbye {
  const LEAVING_MESSAGE = "Thank you for visiting pmc";
}

echo Goodbye::LEAVING_MESSAGE;
?>
```

Or, we can access a constant from inside the class by using the `self` keyword followed by the scope resolution operator (`::`) followed by the constant name, like here:

Example

```
<?php
class Goodbye {
  const LEAVING_MESSAGE = "Thank you for visiting W3Schools.com!";
  public function byebye() {
    echo self::LEAVING_MESSAGE;
  }
}

$goodbye = new Goodbye();
$goodbye->byebye();
```

```
?>
```

PHP - What are Abstract Classes and Methods?

Abstract classes and methods are when the parent class has a named method, but need its child class(es) to fill out the tasks.

An abstract class is a class that contains at least one abstract method. An abstract method is a method that is declared, but not implemented in the code.

An abstract class or method is defined with the `abstract` keyword:

Syntax

```
<?php
abstract class ParentClass {
  abstract public function someMethod1();
  abstract public function someMethod2($name, $color);
  abstract public function someMethod3() : string;
}
?>
```

When inheriting from an abstract class, the child class method must be defined with the same name, and the same or a less restricted access modifier. So, if the abstract method is defined as protected, the child class method must be defined as either protected or public, but not private. Also, the type and number of required arguments must be the same. However, the child classes may have optional arguments in addition.

So, when a child class is inherited from an abstract class, we have the following rules:

- The child class method must be defined with the same name and it redeclares the parent abstract method
- The child class method must be defined with the same or a less restricted access modifier
- The number of required arguments must be the same. However, the child class may have optional arguments in addition

Let's look at an example:

Example

```php
<?php
// Parent class
abstract class Car {
  public $name;
  public function __construct($name) {
    $this->name = $name;
  }
  abstract public function intro() : string;
}

// Child classes
class Audi extends Car {
  public function intro() : string {
    return "Choose German quality! I'm an $this->name!";
  }
}

class Volvo extends Car {
  public function intro() : string {
    return "Proud to be Swedish! I'm a $this->name!";
  }
}

class Citroen extends Car {
  public function intro() : string {
    return "French extravagance! I'm a $this->name!";
  }
}

// Create objects from the child classes
$audi = new audi("Audi");
echo $audi->intro();
echo "<br>";

$volvo = new volvo("Volvo");
echo $volvo->intro();
echo "<br>";

$citroen = new citroen("Citroen");
echo $citroen->intro();
?>
```

Example Explained

The Audi, Volvo, and Citroen classes are inherited from the Car class. This means that the Audi, Volvo, and Citroen classes can use the public $name property as well as the public __construct() method from the Car class because of inheritance.

But, intro() is an abstract method that should be defined in all the child classes and they should return a string.

PHP - More Abstract Class Examples

Let's look at another example where the abstract method has an argument:

Example

```
<?php
abstract class ParentClass {
  // Abstract method with an argument
  abstract protected function prefixName($name);
}

class ChildClass extends ParentClass {
  public function prefixName($name) {
    if ($name == "John Doe") {
      $prefix = "Mr.";
    } elseif ($name == "Jane Doe") {
      $prefix = "Mrs.";
    } else {
      $prefix = "";
    }
    return "{$prefix} {$name}";
  }
}

$class = new ChildClass;
echo $class->prefixName("John Doe");
echo "<br>";
echo $class->prefixName("Jane Doe");
?>
```

Let's look at another example where the abstract method has an argument, and the child class has two optional arguments that are not defined in the parent's abstract method:

Example

```
<?php
abstract class ParentClass {
  // Abstract method with an argument
  abstract protected function prefixName($name);
}

class ChildClass extends ParentClass {
  // The child class may define optional arguments that are not in the
parent's abstract method
  public function prefixName($name, $separator = ".", $greet = "Dear") {
    if ($name == "John Doe") {
      $prefix = "Mr";
    } elseif ($name == "Jane Doe") {
      $prefix = "Mrs";
    } else {
      $prefix = "";
    }
    return "{$greet} {$prefix}{$separator} {$name}";
  }
}

$class = new ChildClass;
echo $class->prefixName("John Doe");
echo "<br>";
echo $class->prefixName("Jane Doe");
?>
```

PHP - What are Interfaces?

Interfaces allow you to specify what methods a class should implement.

Interfaces make it easy to use a variety of different classes in the same way. When one or more classes use the same interface, it is referred to as "polymorphism".

Interfaces are declared with the `interface` keyword:

Syntax

```php
<?php
interface InterfaceName {
  public function someMethod1();
  public function someMethod2($name, $color);
  public function someMethod3() : string;
}
?>
```

PHP - Interfaces vs. Abstract Classes

Interface are similar to abstract classes. The difference between interfaces and abstract classes are:

- Interfaces cannot have properties, while abstract classes can
- All interface methods must be public, while abstract class methods is public or protected
- All methods in an interface are abstract, so they cannot be implemented in code and the abstract keyword is not necessary
- Classes can implement an interface while inheriting from another class at the same time

PHP - Using Interfaces

To implement an interface, a class must use the `implements` keyword.

A class that implements an interface must implement **all** of the interface's methods.

Example

```php
<?php
interface Animal {
  public function makeSound();
}

class Cat implements Animal {
  public function makeSound() {
    echo "Meow";
  }
```

```
}

$animal = new Cat();
$animal->makeSound();
?>
```

From the example above, let's say that we would like to write software which manages a group of animals. There are actions that all of the animals can do, but each animal does it in its own way.

Using interfaces, we can write some code which can work for all of the animals even if each animal behaves differently:

Example

```
<?php
// Interface definition
interface Animal {
  public function makeSound();
}

// Class definitions
class Cat implements Animal {
  public function makeSound() {
    echo " Meow ";
  }
}

class Dog implements Animal {
  public function makeSound() {
    echo " Bark ";
  }
}

class Mouse implements Animal {
  public function makeSound() {
    echo " Squeak ";
  }
}

// Create a list of animals
$cat = new Cat();
$dog = new Dog();
$mouse = new Mouse();
```

```
$animals = array($cat, $dog, $mouse);

// Tell the animals to make a sound
foreach($animals as $animal) {
  $animal->makeSound();
}
?>
```

Example Explained

Cat, Dog and Mouse are all classes that implement the Animal interface, which means that all of them are able to make a sound using the `makeSound()` method. Because of this, we can loop through all of the animals and tell them to make a sound even if we don't know what type of animal each one is.

Since the interface does not tell the classes how to implement the method, each animal can make a sound in its own way.

PHP - What are Traits?

PHP only supports single inheritance: a child class can inherit only from one single parent.

So, what if a class needs to inherit multiple behaviors? OOP traits solve this problem.

Traits are used to declare methods that can be used in multiple classes. Traits can have methods and abstract methods that can be used in multiple classes, and the methods can have any access modifier (public, private, or protected).

Traits are declared with the `trait` keyword:

Syntax

```
<?php
trait TraitName {
  // some code...
}
?>
```

To use a trait in a class, use the `use` keyword:

Syntax

```php
<?php
class MyClass {
  use TraitName;
}
?>
```

Let's look at an example:

Example

```php
<?php
trait message1 {
public function msg1() {
    echo "OOP is fun! ";
  }
}

class Welcome {
  use message1;
}

$obj = new Welcome();
$obj->msg1();
?>
```

Example Explained

Here, we declare one trait: message1. Then, we create a class: Welcome. The class uses the trait, and all the methods in the trait will be available in the class.

If other classes need to use the msg1() function, simply use the message1 trait in those classes. This reduces code duplication, because there is no need to redeclare the same method over and over again.

PHP - Using Multiple Traits

Let's look at another example:

Example

```php
<?php
trait message1 {
  public function msg1() {
    echo "OOP is fun! ";
  }
}

trait message2 {
  public function msg2() {
    echo "OOP reduces code duplication!";
  }
}

class Welcome {
  use message1;
}

class Welcome2 {
  use message1, message2;
}

$obj = new Welcome();
$obj->msg1();
echo "<br>";

$obj2 = new Welcome2();
$obj2->msg1();
$obj2->msg2();
?>
```

Example Explained

Here, we declare two traits: message1 and message2. Then, we create two classes: Welcome and Welcome2. The first class (Welcome) uses the message1 trait, and the second class (Welcome2) uses both message1 and message2 traits (multiple traits are separated by comma).

PHP - Static Methods

Static methods can be called directly - without creating an instance of the class first.

Static methods are declared with the `static` keyword:

Syntax

```php
<?php
class ClassName {
  public static function staticMethod() {
    echo "Hello World!";
  }
}
?>
```

To access a static method use the class name, double colon (::), and the method name:

Syntax

```
ClassName::staticMethod();
```

Let's look at an example:

Example

```php
<?php
class greeting {
  public static function welcome() {
    echo "Hello World!";
  }
}

// Call static method
greeting::welcome();
?>
```

Example Explained

Here, we declare a static method: welcome(). Then, we call the static method by using the class name, double colon (::), and the method name (without creating an instance of the class first).

PHP - More on Static Methods

A class can have both static and non-static methods. A static method can be accessed from a method in the same class using the `self` keyword and double colon (::):

Example

```
<?php
class greeting {
  public static function welcome() {
    echo "Hello World!";
  }

  public function __construct() {
    self::welcome();
  }
}

new greeting();
?>
```

Static methods can also be called from methods in other classes. To do this, the static method should be `public`:

Example

```
<?php
class greeting {
  public static function welcome() {
    echo "Hello World!";
  }
}

class SomeOtherClass {
  public function message() {
    greeting::welcome();
  }
}
?>
```

To call a static method from a child class, use the `parent` keyword inside the child class. Here, the static method can be `public` or `protected`.

Example

```
<?php
class domain {
  protected static function getWebsiteName() {
    return "W3Schools.com";
  }
}

class domainW3 extends domain {
  public $websiteName;
  public function __construct() {
    $this->websiteName = parent::getWebsiteName();
  }
}

$domainW3 = new domainW3;
echo $domainW3 -> websiteName;
?>
```

PHP - Static Properties

Static properties can be called directly - without creating an instance of a class.

Static properties are declared with the `static` keyword:

Syntax

```
<?php
class ClassName {
  public static $staticProp = "W3Schools";
}
?>
```

To access a static property use the class name, double colon (::), and the property name:

Syntax

```
ClassName::$staticProp;
```

Let's look at an example:

Example

```php
<?php
class pi {
  public static $value = 3.14159;
}

// Get static property
echo pi::$value;
?>
```

Example Explained

Here, we declare a static property: $value. Then, we echo the value of the static property by using the class name, double colon (::), and the property name (without creating a class first).

PHP - More on Static Properties

A class can have both static and non-static properties. A static property can be accessed from a method in the same class using the `self` keyword and double colon (::):

Example

```php
<?php
class pi {
  public static $value=3.14159;
  public function staticValue() {
    return self::$value;
  }
}

$pi = new pi();
echo $pi->staticValue();
?>
```

To call a static property from a child class, use the `parent` keyword inside the child class:

Example

```php
<?php
class pi {
  public static $value=3.14159;
}

class x extends pi {
  public function xStatic() {
    return parent::$value;
  }
}

// Get value of static property directly via child class
echo x::$value;

// or get value of static property via xStatic() method
$x = new x();
echo $x->xStatic();
?>
```

PHP Namespaces

Namespaces are qualifiers that solve two different problems:

1. They allow for better organization by grouping classes that work together to perform a task
2. They allow the same name to be used for more than one class

For example, you may have a set of classes which describe an HTML table, such as Table, Row and Cell while also having another set of classes to describe furniture, such as Table, Chair and Bed. Namespaces can be used to organize the classes into two different groups while also preventing the two classes Table and Table from being mixed up.

Declaring a Namespace

Namespaces are declared at the beginning of a file using the `namespace` keyword:

Syntax

Declare a namespace called Html:

```php
<?php
namespace Html;
?>
```

Note: A `namespace` declaration must be the first thing in the PHP file. The following code would be invalid:

```php
<?php
echo "Hello World!";
namespace Html;
...
?>
```

Constants, classes and functions declared in this file will belong to the **Html** namespace:

Example

Create a Table class in the Html namespace:

```php
<?php
namespace Html;
class Table {
  public $title = "";
  public $numRows = 0;
  public function message() {
    echo "<p>Table '{$this->title}' has {$this->numRows} rows.</p>";
  }
}
$table = new Table();
$table->title = "My table";
$table->numRows = 5;
?>

<!DOCTYPE html>
<html>
<body>

<?php
$table->message();
?>

</body>
</html>
```

For further organization, it is possible to have nested namespaces:

Syntax

Declare a namespace called Html inside a namespace called Code:

```
<?php
namespace Code\Html;
?>
```

Using Namespaces

Any code that follows a `namespace` declaration is operating inside the namespace, so classes that belong to the namespace can be instantiated without any qualifiers. To access classes from outside a namespace, the class needs to have the namespace attached to it.

Example

Use classes from the Html namespace:

```
<?php
$table = new Html\Table()
$row = new Html\Row();
?>
```

When many classes from the same namespace are being used at the same time, it is easier to use the `namespace` keyword:

Example

Use classes from the Html namespace without the need for the Html\qualifier:

```
<?php
namespace Html;
$table = new Table();
$row = new Row();
?>
```

Namespace Alias

It can be useful to give a namespace or class an alias to make it easier to write. This is done with the `use` keyword:

Example

Give a namespace an alias:

```
<?php
use Html as H;
$table = new H\Table();
?>
```

Example

Give a class an alias:

```
<?php
use Html\Table as T;
$table = new T();
?>
```

PHP - What is an Iterable?

An iterable is any value which can be looped through with a `foreach()` loop.

The `iterable` pseudo-type was introduced in PHP 7.1, and it can be used as a data type for function arguments and function return values.

PHP - Using Iterables

The `iterable` keyword can be used as a data type of a function argument or as the return type of a function:

Example

Use an iterable function argument:

```
<?php
```

```
function printIterable(iterable $myIterable) {
  foreach($myIterable as $item) {
    echo $item;
  }
}

$arr = ["a", "b", "c"];
printIterable($arr);
?>
```

Example

Return an iterable:

```
<?php
function getIterable():iterable {
  return ["a", "b", "c"];
}

$myIterable = getIterable();
foreach($myIterable as $item) {
  echo $item;
}
?>
```

PHP - Creating Iterables

Arrays

All arrays are iterables, so any array can be used as an argument of a function that requires an iterable.

Iterators

Any object that implements the `Iterator` interface can be used as an argument of a function that requires an iterable.

An iterator contains a list of items and provides methods to loop through them. It keeps a pointer to one of the elements in the list. Each item in the list should have a key which can be used to find the item.

An iterator must have these methods:

- `current()` - Returns the element that the pointer is currently pointing to. It can be any data type
- `key()` Returns the key associated with the current element in the list. It can only be an integer, float, boolean or string
- `next()` Moves the pointer to the next element in the list
- `rewind()` Moves the pointer to the first element in the list
- `valid()` If the internal pointer is not pointing to any element (for example, if next() was called at the end of the list), this should return false. It returns true in any other case

Example

Implement the Iterator interface and use it as an iterable:

```php
<?php
// Create an Iterator
class MyIterator implements Iterator {
  private $items = [];
  private $pointer = 0;

  public function __construct($items) {
    // array_values() makes sure that the keys are numbers
    $this->items = array_values($items);
  }

  public function current() {
    return $this->items[$this->pointer];
  }

  public function key() {
    return $this->pointer;
  }

  public function next() {
    $this->pointer++;
  }

  public function rewind() {
    $this->pointer = 0;
  }

  public function valid() {
    // count() indicates how many items are in the list
```

```
    return $this->pointer < count($this->items);
  }
}

// A function that uses iterables
function printIterable(iterable $myIterable) {
  foreach($myIterable as $item) {
    echo $item;
  }
}

// Use the iterator as an iterable
$iterator = new MyIterator(["a", "b", "c"]);
printIterable($iterator);
?>
```

PHP Anonymous classes

Introduction

As th name suggests, anonymous class is the one which doesn't have name. It is meant for one time use, and if one needs to define a class on the fly. Feature of anonymous class has been introduced since PHP 7 version.

Definition of anonymous class lies inside an expression whose result is an object of that class. It is defined with **new class** syntax as follows

Syntax

```
<?php
$obj=new class {
  public function sayhello(){
    echo "Hello World";
  }
};
$obj->sayhello();
?>
```

Anonymous class can perform everything that a normal can i.e. extend another class, implement interface or use trait

In following code, an anonymous class extends parentclass and implements parentinterface

Example

```
<?php
class parentclass{
  public function test1(){
    echo "test1 method in parent class
";
  }
}
interface parentinterface{
  public function test2();
}
$obj=new class() extends parentclass implements parentinterface {
  public function test2(){
    echo "implements test2 method from interface";
  }
};
$obj->test1();
$obj->test2();
?>
```

Output

The output is as below −

test1 method in parent class

implements test2 method from interface

Nested anonymous class

Anonymous class can be nested inside body of other class method. However, its object doesn't have access to private or protected members of outer class

Example

```
<?php
class testclass{
```

```
  public function test1(){
    return new class(){
      public function test2(){
        echo "test2 method of nested anonymous class";
      }
    };
  }
}
$obj2=new testclass();
$obj2->test1()->test2();
?>
```

Output

Above code produces following result −

test2 method of nested anonymous class

Internal name of anonymous class

PHP parser does give a unique name to anonymous class for internal use

Example

```
<?php
var_dump(get_class(new class() {} ));
?>
```

Output

This will produce output similar to following −

string(60) "class@anonymous/home/cg/root/1569997/main.php0x7f1ba68da026"

Introspection

- Introspection is the ability of a program to examine an object's characteristics, such as its name, parent class (if any), properties, and methods.

- With introspection, you can write code that operates on any class or object. You don't need to know which meth ods or properties are defined when you write your code; instead, you can discover that information at runtime, which makes it possible for you to write generic debug gers, serializers, profilers, and the like. In this section, we look at the introspective functions provided by PHP.

Examining Classes

To determine whether a class exists, use the class_exists() function, which takes in a string and returns a Boolean value.

Alternately, you can use the get_declared_classes() function, which returns an array of defined classes and checks if the class name is in the returned array:

```
$doesClassExist = class_exists(classname);
$classes = get_declared_classes();
$doesClassExist = in_array(classname, $classes);
```

You can get the methods and properties that exist in a class (including those that are inherited from superclasses) using the get_class_methods() and get_class_vars() functions.

These functions take a class name and return an array:

```
$methods = get_class_methods(classname);
$properties = get_class_vars(classname);
```

The class name can be either a variable containing the class name, a bare word, or a quoted string:

```
$class = "Person";
$methods = get_class_methods($class);
$methods = get_class_methods(Person); // same
$methods = get_class_methods("Person"); // same
```

The array returned by get_class_methods() is a simple list of method names. The associative array returned by get_class_vars() maps property names to values and also includes inherited properties.

One quirk of get_class_vars() is that it returns only properties that have default values and are visible in the current scope; there's no way to discover uninitialized properties.

Use get_parent_class() to find a class's parent class:

```
$superclass = get_parent_class(classname);
```

Example 6-1 lists the displayClasses() function, which displays all currently

declared classes and the methods and properties for each.

Example 6-1. Displaying all declared classes

```
function displayClasses() {
$classes = get_declared_classes();
foreach ($classes as $class) {
echo "Showing information about {$class}<br />";
$reflection = new ReflectionClass($class);
$isAnonymous = $reflection->isAnonymous() ? "yes" : "no";
echo "Is Anonymous: {$isAnonymous}<br />";
echo "Class methods:<br />";
$methods = $reflection->getMethods(ReflectionMethod::IS_STATIC);
if (!count($methods)) {
echo "<i>None</i><br />";
}
else {
foreach ($methods as $method) {
echo "<b>{$method}</b>()<br />";
```



```
}
}
echo "Class properties:<br />";
$properties = $reflection->getProperties();
if (!count($properties)) {
echo "<i>None</i><br />";
}
else {
foreach(array_keys($properties) as $property) {
echo "<b>\${$property}</b><br />";
}
}
echo "<hr />";
}
}
```

Examining an Object

To get the class to which an object belongs, first make sure it is an object using the

is_object() function, and then get the class with the get_class() function:

```
$isObject = is_object(var);
$classname = get_class(object);
```

Before calling a method on an object, you can ensure that it exists using the method_exists() function:

```
$methodExists = method_exists(object, method);
```

Calling an undefined method triggers a runtime exception.Just as get_class_vars() returns an array of properties for a class,get_object_vars() returns an array of properties set in an object:

```
$array = get_object_vars(object);
```

And just as get_class_vars() returns only those properties with default values,get_object_vars() returns only those properties that are set:

```
class Person {
public $name;
public $age;
}
$fred = new Person;
$fred->name = "Fred";
$props = get_object_vars($fred); // array('name' => "Fred", 'age' => NULL);
```

The get_parent_class() function accepts either an object or a class name. It returns

the name of the parent class, or FALSE if there is no parent class:

```
class A {}
class B extends A {}
$obj = new B;
echo get_parent_class($obj);
echo get_parent_class(B);
```

A A